Escuela de Ciencias de la Computación
CC4P1 Programación Concurrente y Distribuida · Parcial 2026-II

Open AI Cubes

Red neuronal convolucional concurrente, paralela y distribuida con nodos en Java, Python y Go

Integrantes	[Integrante 1] · [Integrante 2] · [Integrante 3]
Profesor	[Nombre del profesor]
Fecha	Octubre de 2026

Índice

1. Introducción y objetivos
2. Arquitectura del sistema
3. Modelo de procesamiento de imágenes (CNN) y micro chunks
4. Qué se paraleliza y qué se distribuye
5. Protocolo de comunicación entre nodos
6. Desarrollo del programa
7. Evaluación de desempeño (matriz de 12 casos)
8. Pruebas de carga y límites del sistema
9. Despliegue en clúster
10. Conclusiones
11. Anexo: cómo ejecutar

1. Introducción y objetivos

Open AI Cubes es una arquitectura modular en la que cada "cubo" es una capacidad de IA que se despliega en varios nodos y coopera en paralelo. En este trabajo implementamos el cubo de **procesamiento de imágenes**: una red neuronal convolucional (CNN) que se entrena y se consulta de forma concurrente, paralela y distribuida, más un **backend** que atiende consultas de chat y un **frontend de escritorio** donde el usuario pide "Identificar" una imagen y el bot responde con la clase reconocida.

Restricciones del enunciado que cumple el sistema:

- Todo está construido con la **librería estándar** de cada lenguaje: sockets TCP, hilos/procesos/goroutines, `javax.imageio`. No se usan librerías de redes neuronales, ni numpy, ni HTTP, WebSocket, Socket.IO o middleware.
- Clientes y servidor central en **Java (LP1, compatible con JDK 8)**; nodos en **Java (Caso 1)**, **Python (Caso 2, LP2)** y **Go (Caso 3, LP3)**.
- Evaluación en cuatro escenarios (secuencial, paralelo, distribuido y paralelo distribuido) para cada caso de lenguaje.
- Pruebas con cientos de clientes y delay aleatorio para saturar el sistema.

2. Arquitectura del sistema

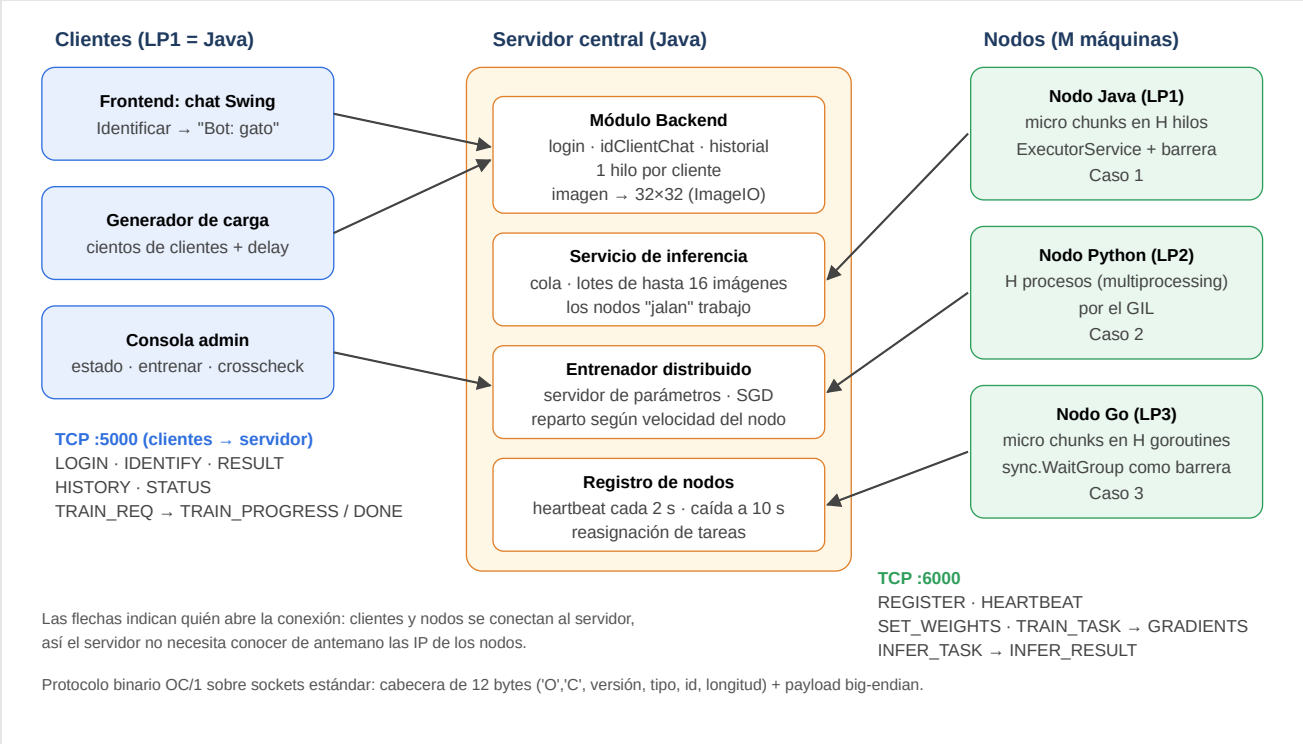


Figura 1. Arquitectura: clientes, servidor central y nodos de cómputo.

El sistema tiene tres tipos de programa. Los **clientes** (chat Swing, generador de carga y consola de administración) se conectan al **servidor central** por el puerto 5000. Los **nodos de cómputo** se conectan al mismo servidor por el puerto 6000. Siempre es el cliente o el nodo quien abre la conexión, por lo que el servidor no necesita conocer de antemano las IP de las máquinas del clúster: basta con que cada nodo conozca la IP del servidor.

Componente	Lenguaje	Responsabilidad
Módulo Backend (en el servidor)	Java	Login y asignación de `idClientChat`, historial del chat (en memoria y en `results/chat_history.txt`), conversión de la imagen recibida a 32×32, estadísticas.
Servicio de inferencia (en el servidor)	Java	Cola de consultas; arma lotes de hasta 16 imágenes; cada nodo tiene 2 hilos que "jalan" lotes, así un nodo más rápido recibe más trabajo. Si no hay nodos, el servidor responde con su propia CNN.
Entrenador distribuido (en el servidor)	Java	Servidor de parámetros: reparte cada lote entre los nodos según su velocidad medida, suma los gradientes y actualiza los pesos con SGD.
Registro de nodos (en el servidor)	Java	Acepta nodos, recibe heartbeats cada 2 s, da de baja a un nodo tras 10 s sin noticias y reasigna sus tareas.
Nodos de cómputo	Java, Python, Go	Ejecutan la CNN (forward y backward) con los micro chunks en H hilos/procesos. No guardan estado aparte de los pesos.
Frontend (chat)	Java Swing	Ventana de chat del enunciado: "Usuario Validado", `idClientChat`, "Yo: Identificar" / "Bot: gato".
Generador de carga	Java	Simula cientos de clientes con delay aleatorio y mide latencia, throughput y errores.

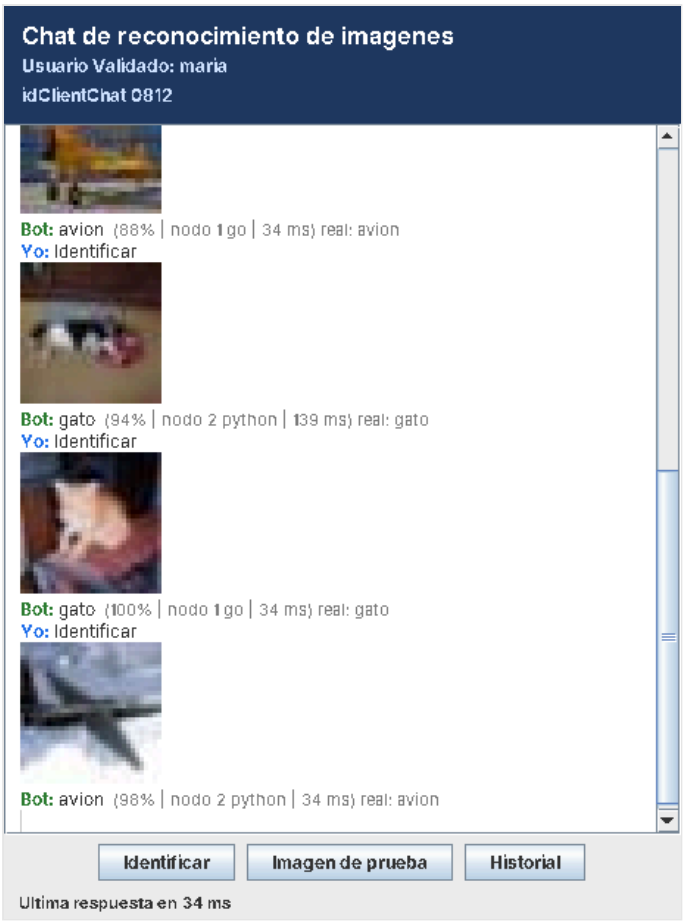


Figura 2. Frontend de escritorio: cada respuesta indica la clase, la confianza, el nodo y el lenguaje que la calculó.

3. Modelo de procesamiento de imágenes (CNN) y micro chunks

3.1 Dataset

Usamos 4 clases de CIFAR-10 (avión, automóvil, gato y barco): 20 000 imágenes de entrenamiento y 4 000 de prueba, de 32×32 píxeles en color. Se guardan en el formato binario de CIFAR (1 byte de etiqueta + 3 072 bytes de píxeles), que cualquier lenguaje puede leer con su librería estándar. La herramienta `DatasetBuilder` convierte carpetas de imágenes JPG/PNG a ese formato, por lo que se pueden agregar clases propias (por ejemplo, fotos de pandas).

3.2 Arquitectura de la red

Capa	Entrada	Salida	Parámetros	Tipo de micro chunk
conv1 3×3 + ReLU	3×32×32	16×32×32	448	Conv (por filtros)
max pool 2×2	16×32×32	16×16×16	0	Pool (por canales)
conv2 3×3 + ReLU	16×16×16	32×16×16	4 640	Conv (por filtros)
max pool 2×2	32×16×16	32×8×8	0	Pool (por canales)
fc1 + ReLU (MLP)	2 048	128	262 272	MLP (por neuronas)
fc2 + softmax (MLP)	128	4	516	MLP (por neuronas)
Total			**267 876**	

La primera parte (convoluciones + pooling) extrae características; la segunda (MLP) clasifica. Se entrena con entropía cruzada y SGD con momentum 0,9, tasa de aprendizaje 0,01 y lotes de 64 imágenes, partiendo siempre de los mismos pesos iniciales (`data/init_weights.bin`, semilla 42) para que todas las mediciones sean comparables.

Resultado del modelo: entrenado 8 épocas sobre las 20 000 imágenes (5 minutos con 4 hilos), alcanza **82,3 % de acierto** en las 4 000 imágenes de prueba (92,5 % en entrenamiento). Los pesos quedan en `data/model.bin` y son los que usa el chat.

3.3 Micro chunks

Un micro chunk es un rango de trabajo independiente dentro de una capa. Rangos distintos escriben en zonas distintas de memoria, así que varios hilos pueden ejecutarlos a la vez sin candados:

Micro chunk	Forward	Backward: gradiente de pesos	Backward: gradiente de entrada
Convolución	rango de filtros	rango de filtros	rango de canales de entrada
MLP	rango de neuronas	rango de neuronas	rango de entradas
Pooling	rango de canales	rango de canales (enruta el gradiente al máximo y aplica la derivada de ReLU)	

El `ChunkExecutor` (Java), su equivalente en Go y el reparto por procesos en Python dividen el rango completo entre H trabajadores y **esperan a que todos terminen** antes de pasar a la siguiente capa. Esa espera es una barrera: la capa siguiente necesita la salida completa de la anterior. El mismo código de micro chunks se usa en el entrenamiento local, en los nodos y en el servidor (respaldo de inferencia), es decir, el módulo de imágenes reutiliza los micro chunks tal como pide el enunciado.

3.4 Verificación de correctitud

- **Gradiente numérico** (`GradCheck`): el gradiente analítico coincide con el de diferencias finitas en todas las capas.
- **Determinismo**: con 1 o 4 hilos la pérdida y el acierto son idénticos, porque cada micro chunk suma en el mismo orden.
- **Prueba cruzada entre lenguajes** (`crosscheck`): el servidor manda las mismas 8 imágenes y los mismos pesos a cada nodo y compara con Java. Resultado: **OK en los 12 nodos**. Go y Java coinciden bit a bit (error 0); Python difiere en menos de $6 \cdot 10^{-6}$ en los gradientes y $6 \cdot 10^{-7}$ en las probabilidades (usa doble precisión internamente).
- **Distribuido = local**: entrenar 1 024 imágenes en el clúster da exactamente la misma pérdida (1,2444) y acierto (43,95 %) que en un solo proceso.

4. Qué se paraleliza y qué se distribuye

Nivel	Qué se reparte	Mecanismo
Distribuido (entre máquinas)	Las imágenes de cada lote (paralelismo de datos). Para el chat, cada consulta o lote de consultas.	Sockets TCP, protocolo OC/1, servidor de parámetros.
Paralelo (dentro de un nodo)	Los micro chunks de cada capa: filtros, neuronas, canales.	Java: `ExecutorService`; Go: goroutines + `sync.WaitGroup`; Python: `multiprocessing` (por el GIL, reparte imágenes entre procesos).
Concurrente (en el servidor)	Muchos clientes y nodos a la vez.	Un hilo por conexión, cola de peticiones, futuros por id de petición, envíos sincronizados por socket.

Un paso de entrenamiento distribuido: (1) el servidor divide el lote de 64 imágenes entre los nodos en proporción a su velocidad medida (imágenes/s, promedio móvil); (2) envía en paralelo a cada nodo los pesos actuales y su parte; (3) cada nodo calcula la *suma* de los gradientes de sus imágenes con sus H hilos; (4) el servidor suma, divide entre 64 y aplica SGD. Como la suma de gradientes no depende de cómo se reparta el lote, el resultado es el mismo que en un solo proceso.

Lo que no se puede paralelizar: las capas van en orden y cada paso de entrenamiento necesita los pesos del paso anterior, así que hay una barrera por capa dentro del nodo y una sincronización por lote entre nodos.

Por qué no repartir capas entre máquinas (paralelismo de modelo): habría que enviar activaciones por la red en cada capa y para cada imagen. Repartiendo imágenes, cada nodo intercambia una sola vez por lote los pesos y gradientes (267 876 floats ≈ 1 MB en cada sentido).

5. Protocolo de comunicación entre nodos

Protocolo binario propio **OC/1** sobre TCP, implementado con `java.net.Socket`, el módulo `socket` de Python y el paquete `net` de Go. Todos los números van en big-endian.

```
+-----+-----+---------+------+-------------+------------+-------------------+
| 'O' | 'C' | versión | tipo | id petición | longitud   | payload           |
| 1 B | 1 B | 1 B     | 1 B  | int32       | int32      | longitud bytes    |
+-----+-----+---------+------+-------------+------------+-------------------+
```

Código	Mensaje	Dirección	Contenido
0x01 / 0x02	REGISTER / REGISTER_OK	nodo ↔ servidor	lenguaje, núcleos, hilos, host / id de nodo
0x03	HEARTBEAT	nodo → servidor	tareas pendientes (cada 2 s)
0x10	SET_WEIGHTS	servidor → nodo	slot (0 = chat, 1 = entrenamiento), versión, 267 876 floats
0x11 / 0x12	TRAIN_TASK / GRADIENTS	servidor ↔ nodo	versión, imágenes y etiquetas / suma de gradientes, pérdida, aciertos, ms
0x20 / 0x21	INFER_TASK / INFER_RESULT	servidor ↔ nodo	imágenes / probabilidades por clase
0x40 / 0x41	LOGIN / LOGIN_OK	cliente ↔ servidor	usuario / idClientChat y clases
0x42 / 0x43	IDENTIFY / RESULT	cliente ↔ servidor	imagen (JPG/PNG o 32×32) / clase, confianza, nodo, lenguaje, ms
0x44–0x56	HISTORY, STATUS, TRAIN_REQ, CROSSCHECK…	cliente ↔ servidor	historial, estado, entrenamiento remoto, prueba cruzada
0x7F	ERROR	ambos	mensaje de error con el id de la petición

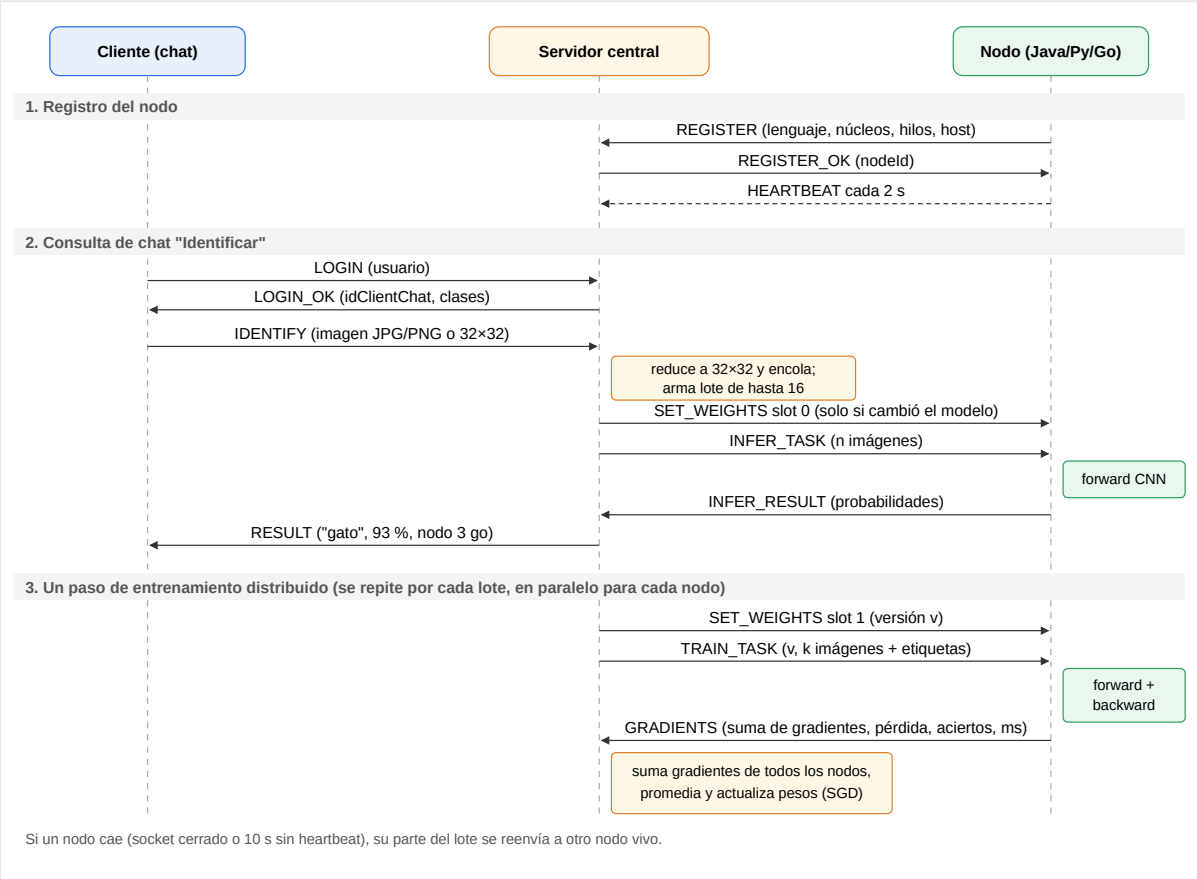


Figura 3. Diagrama de protocolo: registro de un nodo, consulta de chat y paso de entrenamiento.

Tolerancia a fallos: el servidor da por caído a un nodo si se cierra su socket o pasan 10 s sin mensajes; sus peticiones pendientes fallan y se reasignan a otro nodo (en entrenamiento, la parte del lote; en inferencia, el lote vuelve al frente de la cola). Los nodos se reconectan solos cada 3 s

si pierden al servidor. Cada tarea lleva la versión de los pesos; si no coincide, el nodo responde ERROR "version" y el servidor vuelve a enviar los pesos.

6. Desarrollo del programa

Parte	Archivos	Líneas
Java: CNN y micro chunks	9	859
Java: protocolo	4	307
Java: servidor central (backend, inferencia, entrenamiento)	9	1 553
Java: nodo	1	200
Java: clientes (chat, carga, admin)	4	736
Java: benchmark y herramientas	6	900
Python: nodo	1	1 436
Go: nodo	8	1 851

6.1 Java (servidor, clientes y nodo)

Paquetes `cubes.cnn` (CNN y micro chunks), `cubes.net` (tramas y payloads con `DataInputStream`/`ByteBuffer`), `cubes.server`, `cubes.node`, `cubes.client`, `cubes.bench` y `cubes.tools`. Se compila con `javac --release 8`, así que corre en JDK 8 o superior. Las peticiones a los nodos se emparejan con sus respuestas mediante el id de petición y un `CompletableFuture`; un hilo lector por nodo completa los futuros, y los envíos al mismo socket se serializan con un candado.

6.2 Nodo Go (LP3)

Traducción uno a uno de los micro chunks de Java, con float32 y el mismo orden de operaciones, por lo que sus resultados son **idénticos bit a bit** a los de Java. El paralelismo usa goroutines y `sync.WaitGroup` como barrera por capa. Compatible con Go 1.18 o superior y sin módulos externos.

6.3 Nodo Python (LP2)

Mismo diseño de micro chunks, pero adaptado a un lenguaje interpretado sin numpy: un bucle en Python cuesta cerca de 100 veces más que en Java. Por eso la convolución se expresa como **im2col** (tablas de índices aplicadas con `operator.itemgetter`) y los productos punto se hacen en C con `math.dist` / `math.sumprod`; la ReLU se fusiona con el max pooling y los gradientes de la convolución solo recorren las posiciones ganadoras del pooling. Como el **GIL** impide que varios hilos calculen a la vez, el paralelismo usa **H procesos** de `multiprocessing` que viven todo el tiempo del nodo; cada tarea se reparte por imágenes y el proceso principal suma los gradientes. Los pesos llegan a cada proceso una sola vez por versión, por una tubería. Funciona en Python 3.8 o superior, en Linux, Windows y macOS.

6.4 Decisiones de diseño

- **Los nodos se conectan al servidor** y no al revés: se pueden agregar máquinas sin reconfigurar nada.
- **Reparto por velocidad medida:** un nodo Python (más lento) recibe menos imágenes que uno Go o Java, así nadie espera mucho al más lento.

- **Lotes en inferencia:** el servidor agrupa hasta 16 consultas por mensaje; con mucha carga esto aumenta el throughput.
- **Cola acotada (20 000):** si se llena, el servidor responde "servidor saturado" en vez de quedarse sin memoria.
- **Píxeles crudos por la red** (1 byte por canal) en vez de floats: 4 veces menos datos; cada nodo normaliza.

7. Evaluación de desempeño (matriz de 12 casos)

> **Entorno de esta medición:** una sola máquina virtual con 4 núcleos (Intel Xeon 2,8 GHz), 15 GB de RAM, Ubuntu 24.04, OpenJDK 21 (código compilado para Java 8), Python 3.13 y Go 1.24. En los escenarios distribuidos cada nodo es un proceso separado que se comunica por TCP a través de la interfaz local, de modo que los nodos **compiten por los mismos 4 núcleos** y la red no tiene latencia real. Las cifras definitivas deben tomarse en el clúster del laboratorio con los mismos scripts (`bench_local`, `bench_distribuido`, `carga`, `graficas`) y reemplazar estas tablas.

Carga de trabajo: entrenar 1 época sobre 1 024 imágenes, lotes de 64, desde los mismos pesos iniciales. Se mide el tiempo total del entrenamiento (sin la evaluación final).

Escenario	Caso 1: Java	Caso 2: Python	Caso 3: Go / mixto
Secuencial (1 proceso, 1 hilo)	**6,18 s** 165,6 img/s · 1 hilos · java	**55,64 s** 18,4 img/s · 1 hilos · python	**5,62 s** 182,3 img/s · 1 hilos · go
Paralelo (1 máquina, H hilos)	**2,82 s** 362,7 img/s · 4 hilos · java	**17,95 s** 57,0 img/s · 4 hilos · python	**1,88 s** 543,5 img/s · 4 hilos · go
Distribuido (M nodos, 1 hilo c/u)	**2,35 s** 435,7 img/s · 4 nodos × 1 hilos · java	**19,85 s** 51,6 img/s · 4 nodos × 1 hilos · python	**2,96 s** 346,1 img/s · 4 nodos × 1 hilos · java+python+go
Paralelo distribuido (M nodos × H hilos)	**2,91 s** 351,8 img/s · 2 nodos × 2 hilos · java	**18,10 s** 56,6 img/s · 2 nodos × 2 hilos · python	**5,34 s** 191,7 img/s · 2 nodos × 2 hilos · java+python

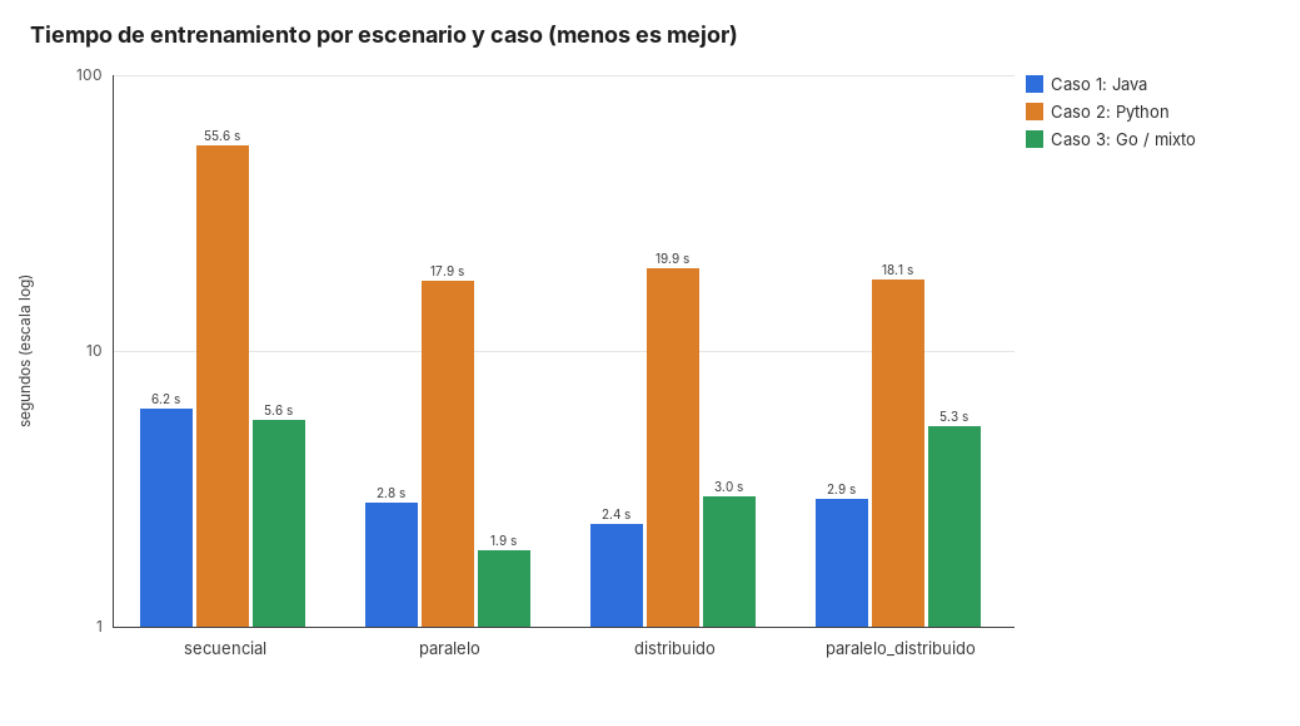


Figura 4. Tiempo por escenario y caso (escala logarítmica).

7.1 Escalabilidad con el número de nodos

Escenario	Lenguaje	Nodos	Hilos/nodo	Segundos	Imágenes/s	Speedup
distribuido	go	1	1	6,41	159,9	1,00×
distribuido	go	2	1	3,41	300,0	1,88×
distribuido	go	4	1	2,39	429,0	2,68×
distribuido	java	1	1	6,40	159,9	1,00×
distribuido	java	2	1	3,96	258,4	1,62×
distribuido	java	4	1	2,35	435,7	2,72×
distribuido	java+python	2	1	7,06	145,1	2,00×
distribuido	java+python+go	4	1	2,96	346,1	4,00×
distribuido	python	1	1	56,27	18,2	1,00×
distribuido	python	2	1	30,89	33,1	1,82×
distribuido	python	4	1	19,85	51,6	2,83×
paralelo distribuido	go	1	2	3,69	277,3	1,00×
paralelo distribuido	go	2	2	2,52	406,4	1,47×
paralelo distribuido	java	1	2	4,48	228,7	1,00×
paralelo distribuido	java	2	2	2,91	351,8	1,54×
paralelo distribuido	java+python	2	2	5,34	191,7	2,00×
paralelo distribuido	python	1	2	32,20	31,8	1,00×
paralelo distribuido	python	2	2	18,10	56,6	1,78×

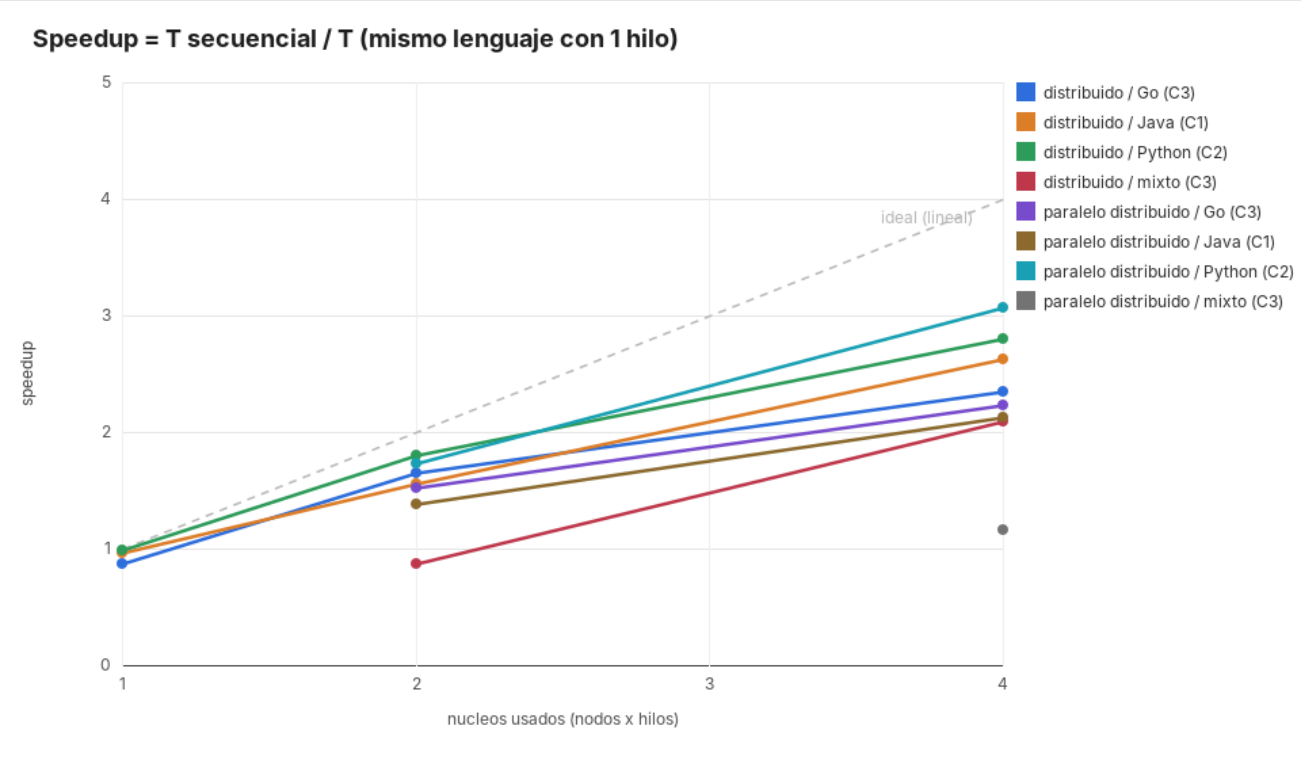


Figura 5. Speedup respecto a 1 nodo.

7.2 Análisis

- **El resultado no depende de cómo se reparta el trabajo.** Las 24 corridas de entrenamiento (local y distribuido, 1 a 4 nodos, 3 lenguajes y clúster mixto) terminan con la misma pérdida (1,2444) y el mismo acierto (43,95 %). Esto confirma que el reparto por imágenes y la suma de gradientes son correctos.
- **Python es unas 10 veces más lento** que Java y Go en cálculo puro (18 img/s contra 166 y 182 con un hilo), porque es interpretado y no usamos numpy. Pero es el que **mejor escala** (3,1× con 4 procesos y 2,8× con 4 nodos): su costo es casi todo cálculo, así que la comunicación pesa poco en proporción.
- **Go escala mejor que Java con hilos en un solo proceso** (2,98× contra 2,19× con 4 hilos). Las goroutines son más livianas y la barrera por capa cuesta menos. En Java, 4 nodos de 1 hilo (2,63×) rinden más que 1 nodo de 4 hilos (2,19×): con lotes de 64 imágenes, las barreras por capa dentro del proceso se notan más que el envío de pesos entre procesos.
- **Con un solo nodo, el distribuido es algo más lento que el secuencial** (Go: 6,4 s contra 5,6 s). La diferencia es el costo de comunicación: en cada lote viajan los pesos (≈1 MB) y la suma de gradientes (≈1 MB).
- **Clúster heterogéneo:** con sincronización por lote, el lote termina cuando termina el nodo más lento. El servidor le da a Python menos imágenes según su velocidad medida, pero Python tiene un costo fijo por lote (preparar los pesos, ≈60 ms) que no se reduce dando menos imágenes. Por eso el mixto Java+Python con 2 nodos (7,1 s) no mejora a Java solo, y el mixto de 4 nodos (3,0 s) queda cerca de Java con 4 nodos (2,4 s). La mejora posible es excluir nodos cuyo costo fijo supere el tiempo del lote, o usar SGD asíncrono.
- **Límite de esta medición:** los speedups no pasan de 3,1× porque todos los nodos comparten 4 núcleos (ley de Amdahl: el servidor suma y actualiza los pesos de forma secuencial, y hay serialización y copia de datos). En máquinas reales cada nodo tiene sus propios núcleos, pero se agrega la red: con 4 nodos se mueven ≈8 MB por lote. En Gigabit son ≈0,07 s por lote; en WiFi o 100 Mbps serían ≈0,7 s y la comunicación dominaría. Por eso se recomienda red cableada Gigabit, o lotes más grandes (`BATCH=256`) para amortizarla.

8. Pruebas de carga y límites del sistema

El generador de carga lanza C clientes simultáneos; cada uno hace login y envía 20 consultas "Identificar" con un delay aleatorio de 0 a 200 ms entre ellas. Se sube C por escalones hasta saturar.

Clientes	Consultas	OK	Errores	Resp/s	Latencia media	p50	p95	p99	Acierto
10	200	200	0	74,1	12,3 ms	4,1	41,6	57,7	77,0 %
50	1000	1000	0	349,2	14,1 ms	7,0	56,9	91,9	79,9 %
100	2000	2000	0	609,0	21,9 ms	13,2	61,5	148,2	80,7 %
200	4000	4000	0	759,9	95,9 ms	70,6	164,9	791,1	82,0 %
400	8000	8000	0	891,3	255,1 ms	214,2	332,3	1 597,1	82,0 %
800	16000	16000	0	905,6	631,4 ms	624,4	844,7	1 986,4	81,9 %

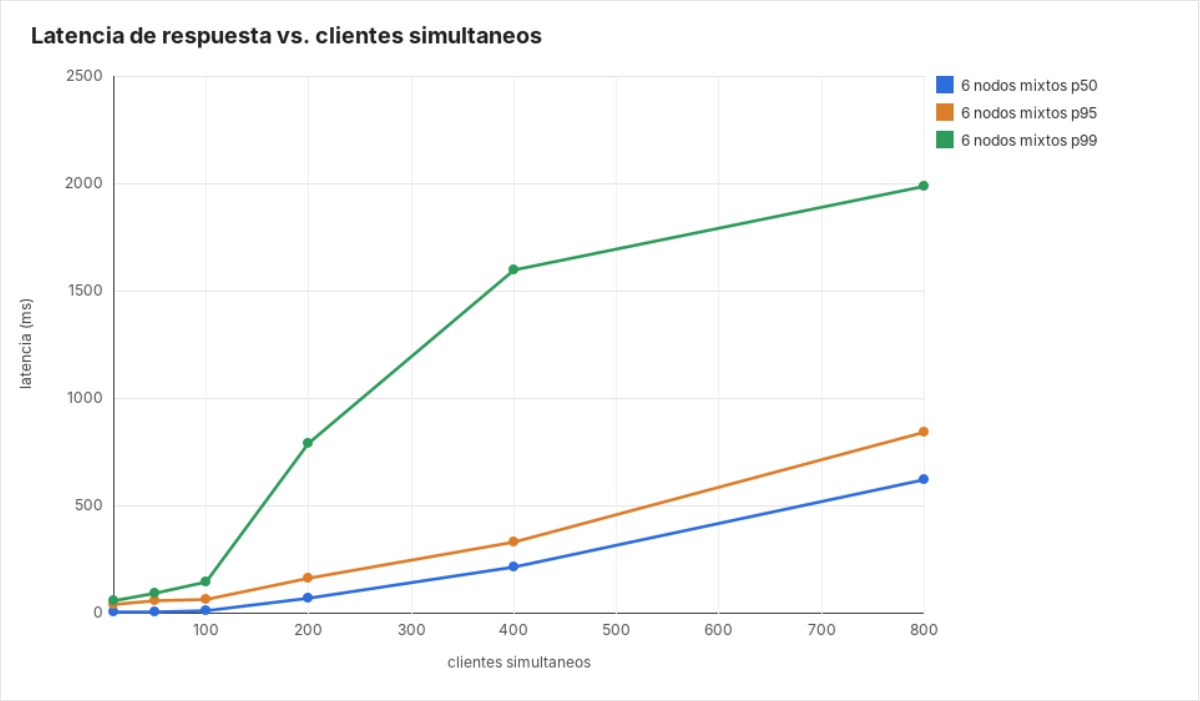


Figura 6. Latencia vs. clientes simultáneos.

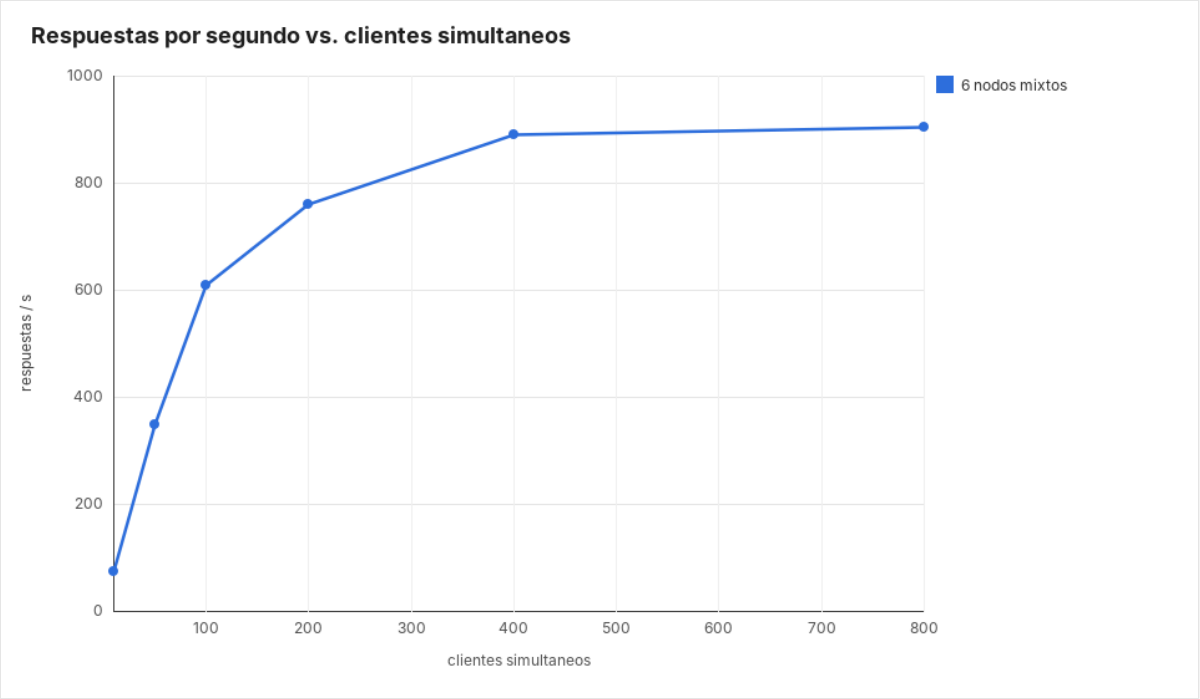


Figura 7. Respuestas por segundo vs. clientes simultáneos.

- **Sin errores ni rechazos** en 31 200 consultas, incluso con 800 clientes simultáneos.
- **Punto de saturación:** el throughput sube casi en proporción a los clientes hasta unos 200 y se estabiliza en **≈900 respuestas/s** desde 400 clientes. A partir de ahí la cola crece y la latencia sube (mediana de 4 ms con 10 clientes, 214 ms con 400 y 624 ms con 800), pero el sistema sigue respondiendo todo.
- El reparto por "jalar" de la cola balancea solo: con poca carga Python atiende el 29 % de las consultas, pero con carga alta solo el 5 %, porque Go y Java vuelven antes por más trabajo.
- Los picos del p99 (≈2 s) corresponden a lotes que tomó el nodo Python cuando la cola estaba llena.

- El cuello de botella aquí es la CPU compartida: el generador de carga, el servidor y los 6 nodos corren en los mismos 4 núcleos. Con más máquinas de cómputo el punto de saturación debería subir en proporción.

9. Despliegue en clúster

1. Copiar el proyecto a todas las máquinas y compilar con `scripts/compilar.sh` (o `.bat` en Windows).
2. Máquina servidor: `scripts/servidor.sh`; abrir los puertos TCP 5000 y 6000 en el firewall.
3. Cada máquina de cómputo: `scripts/nodo_java.sh <ip> <hilos>`, `nodo_python.sh` y `nodo_go.sh`. Los tres pueden quedar conectados; el servidor elige cuáles trabajan según el filtro de lenguaje y prefiere una máquina por nodo.
4. Comprobar con `AdminClient estado` y `AdminClient crosscheck` que todos los nodos responden igual.
5. Correr `bench_local.sh` en una máquina y `bench_distribuido.sh` con 1, 2, 4... máquinas (nodos con 1 hilo y luego con H hilos).
6. Prueba de carga con `carga.sh` y gráficas con `graficas.sh`.

En la nube (opcional): el mismo procedimiento funciona con máquinas virtuales (por ejemplo, el nivel gratuito de un proveedor): el servidor en una VM con IP pública y los puertos 5000/6000 abiertos; los nodos pueden estar en otras VMs o en las computadoras del laboratorio, ya que son ellos quienes abren la conexión.

10. Conclusiones

- Se implementó un sistema distribuido completo, solo con la librería estándar: CNN con micro chunks de convolución y MLP, servidor central con backend, entrenamiento distribuido con servidor de parámetros, inferencia balanceada, chat de escritorio y generador de carga, con un protocolo binario propio sobre TCP.
- Los tres lenguajes (Java, Python y Go) calculan lo mismo, verificado con gradiente numérico, prueba cruzada y entrenamientos idénticos en todos los escenarios.
- El paralelismo dentro del nodo y la distribución entre nodos reducen el tiempo hasta 3× en 4 núcleos. El límite lo ponen la parte secuencial del servidor, la comunicación de pesos y gradientes por lote y, en clústeres heterogéneos, el nodo más lento.
- El servidor atiende cientos de clientes simultáneos sin errores, con un techo de ≈900 respuestas/s en esta máquina, y degrada de forma controlada (sube la latencia, no se cae).
- La elección del lenguaje importa: Python es simple pero 10 veces más lento sin numpy; Go ofrece la mejor relación entre simplicidad y rendimiento para los nodos.

11. Anexo: cómo ejecutar

```
# Compilar (Java para JDK 8 + nodo Go)
scripts/compilar.sh

# Todo en una sola máquina
scripts/demo_local.sh            # servidor + nodo Java + nodo Python + nodo Go +
crosscheck
scripts/cliente.sh               # chat

# Clúster
scripts/servidor.sh                              # máquina servidor
scripts/nodo_java.sh 192.168.1.10 4              # cada máquina de cómputo
scripts/nodo_python.sh 192.168.1.10 4
scripts/nodo_go.sh 192.168.1.10 4
scripts/cliente.sh 192.168.1.10                  # clientes

# Evaluación
scripts/bench_local.sh 1024                      # secuencial y paralelo (Java, Python,
Go)
scripts/bench_distribuido.sh 192.168.1.10 distribuido "1 2 4" 1024
scripts/bench_distribuido.sh 192.168.1.10 paralelo_distribuido "1 2 4" 1024
scripts/carga.sh 192.168.1.10
scripts/graficas.sh
```